ArduPilot PNT (Positioning, Navigation, Timing) Reference Audit

Scope. This document is an audit-grade, read-only static-analysis catalog of every location in the ArduPilot firmware codebase (repository root containing `ArduCopter/, ArduPlane/, Rover/, ArduSub/, AntennaTracker/, Blimp/, libraries/, modules/`) where Positioning, Navigation, and Timing (PNT) data is handled, together with a two-layer dependency map for every catalogued reference. **GROUP 1** catalogs *core* references — code that directly implements, reads, writes, or processes PNT data. **GROUP 2** catalogs *indirect / relational* references — code that does not directly read or write PNT data but whose behavior changes as a result of PNT state. All file paths are repository-root-relative; every code snippet is reproduced verbatim from the cited path and line range so each row is independently verifiable. No source file is modified by this audit.

Legend

Role (Group 1)	**Source** = produces/writes PNT state from a measurement | **Transform** = fuses/derives/converts PNT state (EKF fusion, GPS blending, geodetic↔NED) | **Sink** = only reads PNT state to drive control/behavior.
Group 2 columns	**Trigger Condition** (what invokes the code) | **PNT State Observed** (the exact accessor/value read, e.g. `gps.status()`, `ahrs.have_inertial_nav()`, EKF variance) — the discriminator from Group 1 | **Behavior Change Description** (how behavior changes).
Dependency Type (Layer 1)	**Data dependency** (shared data value/struct field) | **Function call** (direct call) | **Inheritance / Interface** (backend overrides a virtual frontend interface) | **Shared global / singleton** (access via an `AP::` accessor — `AP::gps(), AP::ahrs(), AP::scheduler(), AP::rtc()`).
Notes flags	**[CIRCULAR]** circular dependency (e.g. AHRS↔EKF) | **[SHARED-STRUCT]** PNT fields packed with non-PNT fields (extraction risk) | **[DUPLICATED]** copy-pasted PNT logic across locations | **[MULTI-CATEGORY]** spans >1 PNT category (emitted once per table) | **[ABSENT]** expected PNT element not found where checked.
Cross-reference	Each main table owns a monotonic, per-table # space (1,2,3…). Its dependency sub-table (the matching Xa table) reuses those exact numbers as `Ref #`. Each Xa table contains a **Layer 1** block (direct edges) followed by a **Layer 2** block (full transitive chain + final consumer + hop count).

Footprint surveyed: libraries/ = 151 subsystems; libraries/AP_GPS/.{cpp,h} = 44 files (14 receiver backends + AP_GPS_Blended + AP_GPS_Params); libraries/AP_NavEKF3/*.cpp = 13 fusion modules; flight/drive-mode files = 81 (ArduCopter 29 + ArduPlane 26 + Rover 14 + ArduSub 12). PNT state is produced under libraries/ and consumed/acted-upon in vehicle glue code (failsafe, ekf_check, fence, flight-mode gating).*

GROUP 1 — CORE PNT REFERENCES

Code that **directly implements, reads, writes, or processes** PNT data. Role is one of Source / Transform / Sink.

Table 1 — Core Positioning (GPS, Location, inertial position, position controllers, non-GPS sources)

#	File Path	Line(s)	Function / Class	Role	Code Snippet (5-10 lines)	Notes

1	libraries/AP_GPS/AP_GPS_UBLOX.cpp	1510-1518	AP_GPS_UBLOX::_parse_gps() (NAV-POSLLH)	Source	`_last_pos_time = _buffer.posllh.itow;`
					`state.location.lng = _buffer.posllh.longitude;`
					`state.location.lat = _buffer.posllh.latitude;`
					`state.have_undulation = true;`
					`state.undulation = (_buffer.posllh.altitude_msl - _buffer.posllh.altitude_ellipsoid) * 0.001;`
					`set_alt_amsl_cm(state, _buffer.posllh.altitude_msl / 10);`
					
					`state.status = next_fix;`
					`_new_position = true;`
						Receiver backend writes GPS_State.location.lat/lng and status from the UBX NAV-POSLLH message. Representative of 14 receiver backends (UBLOX, NMEA, SBF, NOVA, SBP/SBP2, SIRF, ERB, MAV, MSP, GSOF, DroneCAN, ExternalAHRS, SITL) — all Sources. [SHARED-STRUCT] target GPS_State.

2	libraries/AP_GPS/GPS_Backend.cpp	101-109	AP_GPS_Backend::fill_3d_velocity()	Transform	`void AP_GPS_Backend::fill_3d_velocity(void)`
					`{`
					`float gps_heading = radians(state.ground_course);`
					
					`state.velocity.x = state.ground_speed * cosf(gps_heading);`
					`state.velocity.y = state.ground_speed * sinf(gps_heading);`
					`state.velocity.z = 0;`
					`state.have_vertical_velocity = false;`
					`}`
						Backend base class; derives state.velocity (NED) from ground_speed + ground_course for receivers that do not report 3D velocity. Inheritance/Interface base shared by all backends.

3	libraries/AP_GPS/AP_GPS.h	324-329	AP_GPS::location()	Source	`const Location &location(uint8_t instance) const {`
					`return state[instance].location;`
					`}`
					`const Location &location() const {`
					`return location(primary_instance);`
					`}`
						Frontend accessor; dispatches to primary_instance. Reached vehicle-side via the AP::gps() singleton.

4	libraries/AP_GPS/AP_GPS.h	360-365	AP_GPS::velocity()	Source	`const Vector3f &velocity(uint8_t instance) const {`
					`return state[instance].velocity;`
					`}`
					`const Vector3f &velocity() const {`
					`return velocity(primary_instance);`
					`}`
						3D NED velocity accessor. [MULTI-CATEGORY] velocity also drives Navigation fusion.

5	libraries/AP_GPS/AP_GPS.h	195-204	struct AP_GPS::GPS_State	Source	`// all the following fields must all be filled by the backend driver`
					`GPS_Status status; ///< driver fix status`
					`uint32_t time_week_ms; ///< GPS time (milliseconds from start of GPS week)`
					`uint16_t time_week; ///< GPS week number`
					`Location location; ///< last fix location`
					`float ground_speed; ///< ground speed in m/s`
					`float ground_course; ///< ground course in degrees, wrapped 0-360`
					`float gps_yaw; ///< GPS derived yaw information, if available (degrees)`
					`uint32_t gps_yaw_time_ms; ///< timestamp of last GPS yaw reading`
					`bool gps_yaw_configured; ///< GPS is configured to provide yaw`
						[SHARED-STRUCT][MULTI-CATEGORY] One struct mixes Positioning (location), Navigation/velocity (velocity, ground_speed, ground_course) and Timing (time_week_ms, time_week) — see Table 3 #10/#11. Primary extraction-risk struct for a PNT-service split.

6	libraries/AP_GPS/AP_GPS_Blended.cpp	17-25	AP_GPS_Blended::_calc_weights()	Transform	`bool AP_GPS_Blended::_calc_weights(void)`
					`{`
					`// zero the blend weights`
					`memset(&_blend_weights, 0, sizeof(_blend_weights));`
					
					`static_assert(GPS_MAX_RECEIVERS == 2, "GPS blending only currently works with 2 receivers");`
					`// Note that the early quit below relies upon exactly 2 instances`
					`// The time delta calculations below also rely upon every instance being currently detected and being parsed`
						Multi-receiver blending of location/velocity (GPS_MAX_RECEIVERS==2). Reads gps.state[].status (Data dependency) to weight receivers.

7	libraries/AP_Common/Location.h	11-20	class Location	Source	`uint8_t relative_alt : 1; // 1 if altitude is relative to home`
					`uint8_t loiter_ccw : 1; // 0 if clockwise, 1 if counter clockwise`
					`uint8_t terrain_alt : 1; // this altitude is above terrain`
					`uint8_t origin_alt : 1; // this altitude is above ekf origin`
					`uint8_t loiter_xtrack : 1; // 0 to crosstrack from center of waypoint, 1 to crosstrack from tangent exit location`
					
					`// note that mission storage only stores 24 bits of altitude (~ +/- 83km)`
					`int32_t alt; // in cm`
					`int32_t lat; // in 1E7 degrees`
					`int32_t lng; // in 1E7 degrees`
						[SHARED-STRUCT] Geodetic PNT fields lat/lng (1E7 deg) and alt (cm) packed with NON-PNT bitfields relative_alt, loiter_ccw, terrain_alt, origin_alt, loiter_xtrack. Ubiquitous value type; top extraction risk.

#	File Path	Line(s)	Function / Class	Role	Code Snippet (5-10 lines)	Notes

8	libraries/AP_Common/Location.cpp	259-268	Location::get_vector_from_origin_NEU_cm()	Transform	`bool Location::get_vector_from_origin_NEU_cm(T &vec_neu) const`
					`{`
					`  // convert altitude`
					`  int32_t alt_above_origin_cm = 0;`
					`  if (!get_alt_cm(AltFrame::ABOVE_ORIGIN, alt_above_origin_cm)) {`
					`  return false;`
					`  }`
					`  // convert lat, lon`
					`  if (!get_vector_xy_from_origin_NE_cm(vec_neu.xy())) {`
						Geodetic (lat/lng/alt) → NEU centimetre vector relative to the EKF origin (geodetic↔NED duality).

9	libraries/AP_InertialNav/AP_InertialNav.cpp	14-23	AP_InertialNav::update()	Transform	`void AP_InertialNav::update(bool high_vibes)`
					`{`
					`  // get the NE position relative to the local earth frame origin`
					`  Vector2f posNE;`
					`  if (_ahrs_ekf.get_relative_position_NE_origin_float(posNE)) {`
					`  _relpos_cm.x = posNE.x * 100; // convert from m to cm`
					`  _relpos_cm.y = posNE.y * 100; // convert from m to cm`
					`  }`
					`  // get the D position relative to the local earth frame origin`
						Pulls EKF origin-relative NE/D position into _relpos_cm (cm). get_position_neu_cm() (L53-56) returns it as a Sink/accessor for vehicle code.

10	libraries/AC_AttitudeControl/AC_PosControl.cpp	363-372	AC_PosControl::input_pos_NEU_cm()	Sink	`void AC_PosControl::input_pos_NEU_cm(const Vector3p& pos_neu_cm, float pos_terrain_target_u_cm, float terrain_buffer_cm)`
					`{`
					`  input_pos_NEU_m(pos_neu_cm * 0.01, pos_terrain_target_u_cm * 0.01, terrain_buffer_cm * 0.01);`
					`}`
					`// Sets a new NEU position target in meters and computes a jerk-limited trajectory.`
					`// Updates internal acceleration commands using a smooth kinematic path constrained`
					`// by configured acceleration and jerk limits. Terrain margin is used to constrain`
					`// horizontal velocity to avoid vertical buffer violation.`
					`void AC_PosControl::input_pos_NEU_m(const Vector3p& pos_neu_m, float pos_terrain_target_u_m, float terrain_buffer_m)`
						3D position/velocity controller consuming the position target. [ABSENT] there is no standalone AC_PosControl library — it physically resides under libraries/AC_AttitudeControl/ (compatibility note).

11	libraries/APM_Control/AR_PosControl.cpp	114-123	AR_PosControl::update()	Sink	`void AR_PosControl::update(float dt)`
					`{`
					`  // exit immediately if no current location, destination or disarmed`
					`  Vector3p curr_pos_NED_m;`
					`  Vector3f curr_vel_NED;`
					`  if (!hal.util->get_soft_armed() || !AP::ahrs().get_relative_position_NED_origin(curr_pos_NED_m) ||`
					`  !AP::ahrs().get_velocity_NED(curr_vel_NED)) {`
					`  _desired_speed = _atc.get_desired_speed_accel_limited(0.0f, dt);`
					`  _desired_lat_accel = 0.0f;`
					`  _desired_turn_rate_rads = 0.0f;`
						Rover position controller; reads AP::ahrs().get_relative_position_NED_origin() and get_velocity_NED() (Shared global/singleton). Separate controller from Copter's AC_PosControl.

12	libraries/AP_Beacon/AP_Beacon.cpp	175-184	AP_Beacon::get_vehicle_position_ned()	Source	`bool AP_Beacon::get_vehicle_position_ned(Vector3f &position, float& accuracy_estimate) const`
					`{`
					`  if (!device_ready()) {`
					`  return false;`
					`  }`
					`  // check for timeout`
					`  if (AP_HAL::millis() - veh_pos_update_ms > AP_BEACON_TIMEOUT_MS) {`
					`  return false;`
					`  }`
						Non-GPS (GPS-denied) positioning source; returns vehicle NED position with a staleness timeout.

13	libraries/AP_VisualOdom/AP_VisualOdom.cpp	201-210	AP_VisualOdom::handle_pose_estimate()	Source	`void AP_VisualOdom::handle_pose_estimate(uint64_t remote_time_us, uint32_t time_ms, float x, float y, float z, float roll, float pitch, float yaw, float posErr, float angErr, uint8_t reset_counter, int8_t quality)`
					`{`
					`  // exit immediately if not enabled`
					`  if (!enabled()) {`
					`  return;`
					`  }`
					`  // call backend`
					`  if (_driver != nullptr) {`
					`  // convert attitude to quaternion and call backend`
						Non-GPS visual-odometry position/attitude source feeding EKF external-nav fusion.

Table 1a — Positioning dependencies (Layer 1 direct edges + Layer 2 transitive chains)

Layer 1 — Direct calls / called-by with typed dependency

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

1	AP_GPS_UBLOX backend	fill_3d_velocity(), set_alt_amsl_cm(), _check_new_itow()	AP_GPS::update() → backend read()	Inheritance / Interface	`_last_pos_time = _buffer.posllh.itow;`
					`state.location.lng = _buffer.posllh.longitude;`
					`state.location.lat = _buffer.posllh.latitude;`
					`state.have_undulation = true;`
					`state.undulation = (_buffer.posllh.altitude_msl - _buffer.posllh.altitude_ellipsoid) * 0.001;`
					`set_alt_amsl_cm(state, _buffer.posllh.altitude_msl / 10);`
					
					`state.status = next_fix;`
					`_new_position = true;`

2	AP_GPS_Backend::fill_3d_velocity()	cosf/sinf on state.ground_course; writes state.velocity	All receiver backends (UBLOX, NMEA, SIRF, ...)	Data dependency	`void AP_GPS_Backend::fill_3d_velocity(void)`
					`{`
					`float gps_heading = radians(state.ground_course);`
					
					`state.velocity.x = state.ground_speed * cosf(gps_heading);`
					`state.velocity.y = state.ground_speed * sinf(gps_heading);`
					`state.velocity.z = 0;`
					`state.have_vertical_velocity = false;`
					`}`

3	AP_GPS::location()	state[instance].location	NavEKF3 readGpsData(); AP_AHRS; AC_WPNav::get_vector_NEU_cm	Shared global / singleton	`const Location &location(uint8_t instance) const {`
					`return state[instance].location;`
					`}`
					`const Location &location() const {`
					`return location(primary_instance);`
					`}`

4	AP_GPS::velocity()	state[instance].velocity	NavEKF3 PosVel fusion (readGpsData)	Shared global / singleton	`const Vector3f &velocity(uint8_t instance) const {`
					`return state[instance].velocity;`
					`}`
					`const Vector3f &velocity() const {`
					`return velocity(primary_instance);`
					`}`

5	AP_GPS::GPS_State (struct)	(data struct — no calls)	All GPS backends (write); frontend accessors (read)	Data dependency	`// all the following fields must all be filled by the backend driver`
					`GPS_Status status; ///< driver fix status`
					`uint32_t time_week_ms; ///< GPS time (milliseconds from start of GPS week)`
					`uint16_t time_week; ///< GPS week number`
					`Location location; ///< last fix location`
					`float ground_speed; ///< ground speed in m/s`
					`float ground_course; ///< ground course in degrees, wrapped 0-360`
					`float gps_yaw; ///< GPS derived yaw information, if available (degrees)`
					`uint32_t gps_yaw_time_ms; ///< timestamp of last GPS yaw reading`
					`bool gps_yaw_configured; ///< GPS is configured to provide yaw`

6	AP_GPS_Blended::_calc_weights()	gps.state[].status; time deltas	AP_GPS_Blended::calc_weights()/update (blended primary)	Data dependency	`bool AP_GPS_Blended::_calc_weights(void)`
					`{`
					`// zero the blend weights`
					`memset(&_blend_weights, 0, sizeof(_blend_weights));`
					
					`static_assert(GPS_MAX_RECEIVERS == 2, "GPS blending only currently works with 2 receivers");`
					`// Note that the early quit below relies upon exactly 2 instances`
					`// The time delta calculations below also rely upon every instance being currently detected and being parsed`

7	class Location	get_alt_cm(); get_vector_xy_from_origin_NE_cm()	GPS_State; AP_AHRS; AC_WPNav; AP_Mission (ubiquitous)	Data dependency	`uint8_t relative_alt : 1; // 1 if altitude is relative to home`
					`uint8_t loiter_ccw : 1; // 0 if clockwise, 1 if counter clockwise`
					`uint8_t terrain_alt : 1; // this altitude is above terrain`
					`uint8_t origin_alt : 1; // this altitude is above ekf origin`
					`uint8_t loiter_xtrack : 1; // 0 to crosstrack from center of waypoint, 1 to crosstrack from tangent exit location`
					
					`// note that mission storage only stores 24 bits of altitude (~ +/- 83km)`
					`int32_t alt; // in cm`
					`int32_t lat; // in 1E7 degrees`
					`int32_t lng; // in 1E7 degrees`

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet
8	Location::get_vector_from_origin_NEU_cm()	get_alt_cm(); get_vector_xy_from_origin_NE_cm()	AC_WPNav::get_vector_NEU_cm()	Function call	`bool Location::get_vector_from_origin_NEU_cm(T &vec_neu) const { // convert altitude int32_t alt_above_origin_cm = 0; if (!get_alt_cm(AltFrame::ABOVE_ORIGIN, alt_above_origin_cm)) { return false; } // convert lat, lon if (!get_vector_xy_from_origin_NE_cm(vec_neu.xy())) {`
9	AP_InertialNav::update()	_ahrs_ekf.get_relative_position_NE_origin_float()	Copter::read_inertia()	Function call	`void AP_InertialNav::update(bool high_vibes) { // get the NE position relative to the local earth frame origin Vector2f posNE; if (_ahrs_ekf.get_relative_position_NE_origin_float(posNE)) { _relpos_cm.x = posNE.x * 100; // convert from m to cm _relpos_cm.y = posNE.y * 100; // convert from m to cm } // get the D position relative to the local earth frame origin`
10	AC_PosControl::input_pos_NEU_cm()	input_pos_NEU_m()	ArduCopter mode_auto.cpp (L1463), mode.cpp (L741/L835)	Function call	`void AC_PosControl::input_pos_NEU_cm(const Vector3p& pos_neu_cm, float pos_terrain_target_u_cm, float terrain_buffer_cm) { input_pos_NEU_m(pos_neu_cm * 0.01, pos_terrain_target_u_cm * 0.01, terrain_buffer_cm * 0.01); } // Sets a new NEU position target in meters and computes a jerk-limited trajectory. // Updates internal acceleration commands using a smooth kinematic path constrained // by configured acceleration and jerk limits. Terrain margin is used to constrain // horizontal velocity to avoid vertical buffer violation. void AC_PosControl::input_pos_NEU_m(const Vector3p& pos_neu_m, float pos_terrain_target_u_m, float terrain_buffer_m)`
11	AR_PosControl::update()	AP::ahrs().get_relative_position_NED_origin(); get_velocity_NED()	Rover mode controllers	Shared global / singleton	`void AR_PosControl::update(float dt) { // exit immediately if no current location, destination or disarmed Vector3p curr_pos_NED_m; Vector3f curr_vel_NED; if (!hal.util->get_soft_armed() || !AP::ahrs().get_relative_position_NED_origin(curr_pos_NED_m) || !AP::ahrs().get_velocity_NED(curr_vel_NED)) { _desired_speed = _atc.get_desired_speed_accel_limited(0.0f, dt); _desired_lat_accel = 0.0f; _desired_turn_rate_rads = 0.0f;`
12	AP_Beacon::get_vehicle_position_ned()	device_ready(); AP_HAL::millis()	NavEKF3 range-beacon fusion	Function call	`bool AP_Beacon::get_vehicle_position_ned(Vector3f &position, float& accuracy_estimate) const { if (!device_ready()) { return false; } // check for timeout if (AP_HAL::millis() - veh_pos_update_ms > AP_BEACON_TIMEOUT_MS) { return false; }`
13	AP_VisualOdom::handle_pose_estimate()	_driver backend handle_pose_estimate()	GCS / AP_ExternalAHRS pose handlers	Inheritance / Interface	`void AP_VisualOdom::handle_pose_estimate(uint64_t remote_time_us, uint32_t time_ms, float x, float y, float z, float roll, float pitch, float yaw, float posErr, float angErr, uint8_t reset_counter, int8_t quality) { // exit immediately if not enabled if (!enabled()) { return; } // call backend if (_driver != nullptr) { // convert attitude to quaternion and call backend`

Layer 2 — Full transitive chain to final consumer (hop count)

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
1	AP_GPS_UBLOX (state.location)	AP_GPS_UBLOX → AP_GPS::location() → NavEKF3 readGpsData/FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors	AP_Motors / SRV_Channel	**7**	Sensor source to actuator.
2	GPS velocity (fill_3d_velocity)	GPS_State.velocity → AP_GPS::velocity() → NavEKF3 PosVel fusion → AP_AHRS::get_velocity_NED() → AC_PosControl → AP_Motors	AP_Motors	**5**	[MULTI-CATEGORY] also Navigation.
3	AP_GPS::location()	AP_GPS::location() → NavEKF3 → AP_AHRS::get_location() → AC_WPNav::set_wp_destination_loc → AC_PosControl → AP_Motors	AP_Motors	**5**	Geodetic position path.
4	AP_GPS::velocity()	AP_GPS::velocity() → NavEKF3 PosVel → AP_AHRS → controllers → AP_Motors	AP_Motors	**4**	Velocity path.
5	GPS_State (struct)	GPS_State → AP_GPS accessors {location|velocity|time_week} → EKF / AHRS / AP_RTC → controllers + clock	AP_Motors and AP_RTC	**6**	[SHARED-STRUCT][MULTI-CATEGORY] branches into Table 3.
6	AP_GPS_Blended	Blended → AP_GPS primary (blended) location/velocity → NavEKF3 → AP_AHRS → controllers → AP_Motors	AP_Motors	**6**	Transform upstream of EKF.
7	class Location (value type)	Location → {GPS_State | AHRS | AC_WPNav | AP_Mission} → controllers	AP_Motors / mission logic	**var.**	[SHARED-STRUCT] pervasive value type.
8	Location→NEU conversion	get_vector_from_origin_NEU_cm() → AC_WPNav::get_vector_NEU_cm → set_wp_destination_NEU_cm → AC_PosControl	AC_PosControl	**3**	Geodetic↔NED transform edge.
9	AP_InertialNav	EKF origin-relative pos → AP_InertialNav::update → _relpos_cm → Copter logging/reporting	GCS / logger	**3**	Reporting branch.
10	AC_PosControl (Sink)	AP_AHRS pos → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors	AP_Motors	**4**	Inbound chain to a Sink endpoint.
11	AR_PosControl (Sink, Rover)	AP_AHRS pos/vel → AR_PosControl → AR_AttitudeControl → SRV_Channels	SRV_Channels	**3**	Rover actuation endpoint.
12	AP_Beacon (Source)	AP_Beacon → get_vehicle_position_ned → NavEKF3 RngBcn fusion → AP_AHRS → AP_Motors	AP_Motors	**4**	GPS-denied source.
13	AP_VisualOdom (Source)	AP_VisualOdom → NavEKF3 external-nav fusion → AP_AHRS → AP_Motors	AP_Motors	**4**	GPS-denied source.

Table 2 — Core Navigation (AHRS hub, EKF fusion, waypoint/mission nav, fixed-wing guidance & energy, attitude sink)

#	File Path	Line(s)	Function / Class	Role	Code Snippet (5-10 lines)	Notes

1	libraries/AP_AHRS/AP_AHRS.cpp	3600-3604	AP_AHRS::get_location()	Transform	`bool AP_AHRS::get_location(Location &loc) const`
					`{`
					`  loc = state.location;`
					`  return state.location_ok;`
					`}`
						Central navigation hub returns the fused Location from the active EKF. Reached via AP::ahrs(). [CIRCULAR] AHRS owns NavEKF2/NavEKF3 (AP_AHRS.h L483/L486) while the EKF reads AHRS/sensor state back through AP_DAL (AP_NavEKF3_core.cpp L8/L11) — AHRS↔EKF mutual reference.

2	libraries/AP_AHRS/AP_AHRS.h	240-244	AP_AHRS::groundspeed()	Source	`// return ground speed estimate in meters/second. Used by ground vehicles.`
					`float groundspeed(void) const { return state.ground_speed; }`
					
					`const Vector3f &get_accel_ef() const {`
						Returns fused ground-speed estimate (m/s); used by ground vehicles and reporting.

3	libraries/AP_AHRS/AP_AHRS.h	269-277	AP_AHRS::get_origin() / have_inertial_nav()	Source	`// returns the inertial navigation origin in lat/lon/alt`
					`bool get_origin(Location &ret) const WARN_IF_UNUSED;`
					
					`bool have_inertial_nav() const;`
					
					`// return a ground velocity in meters/second, North/East/Down`
					`// order. Must only be called if have_inertial_nav() is true`
					`bool get_velocity_NED(Vector3f &vec) const WARN_IF_UNUSED;`
						Inertial-nav origin + availability. have_inertial_nav() (L273) is a user-cited accessor and the gate used by vehicle position-health checks (see Table 5 #7).

4	libraries/AP_AHRS/AP_AHRS.cpp	1739-1748	AP_AHRS::get_relative_position_NED_origin()	Transform	`bool AP_AHRS::get_relative_position_NED_origin(Vector3p &vec) const`
					`{`
					`  switch (active_EKF_type()) {`
					`#if AP_AHRS_DCM_ENABLED`
					`  case EKFType::DCM:`
					`  return dcm.get_relative_position_NED_origin(vec);`
					`#endif`
					`#if HAL_NAVEKF2_AVAILABLE`
					`  case EKFType::TWO: {`
					`  Vector2p posNE;`
						Geodetic↔NED: origin-relative NED position via active-EKF-type dispatch (DCM/EKF2/EKF3).

5	libraries/AP_AHRS/AP_AHRS.cpp	3662-3666	AP_AHRS::get_velocity_NED()	Transform	`bool AP_AHRS::get_velocity_NED(Vector3f &vec) const`
					`{`
					`  vec = state.velocity_NED;`
					`  return state.velocity_NED_ok;`
					`}`
						Returns fused NED velocity; consumed by crash detection and Rover position control.

6	libraries/AP_NavEKF3/AP_NavEKF3_core.cpp	627-636	NavEKF3_core::UpdateFilter()	Transform	`void NavEKF3_core::UpdateFilter(bool predict)`
					`{`
					`  // don't run filter updates if states have not been initialised`
					`  if (!statesInitialised) {`
					`  return;`
					`  }`
					
					`  fill_scratch_variables();`
					
					`  // update sensor selection (for affinity)`
						Extended Kalman Filter predict/fuse step (the core PNT transform). [CIRCULAR] EKF reads AHRS/sensors via AP_DAL while AHRS owns the EKF. One of 13 EKF3 fusion modules; EKF2/EKF/NavEKF also present.

7	libraries/AP_NavEKF3/AP_NavEKF3_PosVelFusion.cpp	502-511	NavEKF3_core::SelectVelPosFusion()	Transform	`void NavEKF3_core::SelectVelPosFusion()`
					`{`
					`  // Check if the magnetometer has been fused on that time step and the filter is running at faster than 200 Hz`
					`  // If so, don't fuse measurements on this time step to reduce frame over-runs`
					`  // Only allow one time slip to prevent high rate magnetometer data preventing fusion of other measurements`
					`  if (magFusePerformed && dtIMUavg < 0.005f && !posVelFusionDelayed) {`
					`  posVelFusionDelayed = true;`
					`  return;`
					`  } else {`
					`  posVelFusionDelayed = false;`
						GPS position/velocity fusion scheduling. The dtIMUavg<0.005 gate couples EKF fusion cadence to the scheduler loop rate (timing coupling). [CIRCULAR] AHRS↔EKF.

8	libraries/AC_WPNav/AC_WPNav.cpp	245-254	AC_WPNav::set_wp_destination_loc()	Transform	`bool AC_WPNav::set_wp_destination_loc(const Location& destination)`
					`{`
					`  bool is_terrain_alt;`
					`  Vector3f dest_neu_cm;`
					
					`  // convert destination location to vector`
					`  if (!get_vector_NEU_cm(destination, dest_neu_cm, is_terrain_alt)) {`
					`  return false;`
					`  }`
						Waypoint navigation: converts a Location waypoint to a NEU target (get_vector_NEU_cm) relative to EKF origin.

#	File Path	Line(s)	Function / Class	Role	Code Snippet (5-10 lines)	Notes

9	libraries/AP_Mission/AP_Mission.cpp	552-561	AP_Mission::get_next_nav_cmd()	Source	`bool AP_Mission::get_next_nav_cmd(uint16_t start_index, Mission_Command& cmd)`
					`{`
					`  // search until the end of the mission command list`
					`  for (uint16_t cmd_index = start_index; cmd_index < (unsigned)_cmd_total; cmd_index++) {`
					`  // get next command`
					`  if (!get_next_cmd(cmd_index, cmd, false)) {`
					`  // no more commands so return failure`
					`  return false;`
					`  }`
					`  // if found a "navigation" command then return it`
						Mission/waypoint storage read and navigation-command sequencing.

10	libraries/AP_L1_Control/AP_L1_Control.cpp	206-215	AP_L1_Control::update_waypoint()	Transform	`void AP_L1_Control::update_waypoint(const Location &prev_WP, const Location &next_WP, float dist_min)`
					`{`
					`  Location _current_loc;`
					`  float Nu;`
					`  float xtrackVel;`
					`  float ltrackVel;`
					`  uint32_t now = AP_HAL::micros();`
					`  float dt = (now - _last_update_waypoint_us) * 1.0e-6f;`
						Fixed-wing lateral (L1) guidance; reads _ahrs and AP_HAL::micros() for dt (timing coupling).

11	libraries/AP_L1_Control/AP_L1_Control.cpp	79-88	AP_L1_Control::nav_roll_cd()	Transform	`int32_t AP_L1_Control::nav_roll_cd(void) const`
					`{`
					`  float ret;`
					`  /*`
					`  formula can be obtained through equations of balanced spiral:`
					`  liftForce * cos(roll) = gravityForce * cos(pitch);`
					`  liftForce * sin(roll) = gravityForce * lateralAcceleration / gravityAcceleration; // as mass = gravityForce/gravityAcceleration`
					`  see issue 24319 [https://github.com/ArduPilot/ardupilot/issues/24319]`
					`  Multiplier 100.0f is for converting degrees to centidegrees`
					`  Made changes to avoid zero division as proposed by Andrew Tridgell: https://github.com/ArduPilot/ardupilot/pull/24331#discussion_r1267798397`
						Derives demanded roll (centi-deg) from lateral-acceleration demand and AHRS pitch.

12	libraries/AP_TECS/AP_TECS.cpp	1259-1268	AP_TECS::update_pitch_throttle()	Transform	`void AP_TECS::update_pitch_throttle(int32_t hgt_dem_cm,`
					`  int32_t EAS_dem_cm,`
					`  enum AP_FixedWing::FlightStage flight_stage,`
					`  float distance_beyond_land_wp,`
					`  int32_t ptchMinCO_cd,`
					`  int16_t throttle_nudge,`
					`  float hgt_afe,`
					`  float load_factor,`
					`  float pitch_trim_deg)`
					`{`
						Fixed-wing Total Energy Control (speed/altitude) producing pitch & throttle demands.

13	libraries/AC_AttitudeControl/AC_AttitudeControl.cpp	411-419	AC_AttitudeControl::input_euler_angle_roll_pitch_yaw_cd()	Sink	`void AC_AttitudeControl::input_euler_angle_roll_pitch_yaw_cd(float euler_roll_angle_cd, float euler_pitch_angle_cd, float euler_yaw_angle_cd, bool slew_yaw)`
					`{`
					`  // Convert from centidegrees on public interface to radians`
					`  const float euler_roll_angle_rad = cd_to_rad(euler_roll_angle_cd);`
					`  const float euler_pitch_angle_rad = cd_to_rad(euler_pitch_angle_cd);`
					`  const float euler_yaw_angle_rad = cd_to_rad(euler_yaw_angle_cd);`
					`  input_euler_angle_roll_pitch_yaw_rad(euler_roll_angle_rad, euler_pitch_angle_rad, euler_yaw_angle_rad, slew_yaw);`
					`}`
						Attitude/rate controller consuming the navigation demand; downstream of all navigation.

Table 2a — Navigation dependencies (Layer 1 direct edges + Layer 2 transitive chains)

Layer 1 — Direct calls / called-by with typed dependency

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

1	AP_AHRS::get_location()	EKF3 getLLH() / state.location	ArduCopter inertia.cpp L14; commands.cpp L27/L43; GCS_MAVLink_Copter.cpp L1354	Shared global / singleton	`bool AP_AHRS::get_location(Location &loc) const`
					`{`
					`  loc = state.location;`
					`  return state.location_ok;`
					`}`

2	AP_AHRS::groundspeed()	state.ground_speed	Rover speed control; GCS reporting	Data dependency	`  // return ground speed estimate in meters/second. Used by ground vehicles.`
					`  float groundspeed(void) const { return state.ground_speed; }`
					`  const Vector3f &get_accel_ef() const {`

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

3	AP_AHRS::have_inertial_nav()/get_origin()	active_EKF_type(); EKF origin	Copter::ekf_has_absolute_position (system.cpp L240); ekf_check	Function call	`// returns the inertial navigation origin in lat/lon/alt`
					`bool get_origin(Location &ret) const WARN_IF_UNUSED;`
					
					`bool have_inertial_nav() const;`
					
					`// return a ground velocity in meters/second, North/East/Down`
					`// order. Must only be called if have_inertial_nav() is true`
					`bool get_velocity_NED(Vector3f &vec) const WARN_IF_UNUSED;`

4	AP_AHRS::get_relative_position_NED_origin()	dcm/EKF2/EKF3 get_relative_position_NED_origin()	AR_PosControl::update; AP_InertialNav::update	Inheritance / Interface	`bool AP_AHRS::get_relative_position_NED_origin(Vector3p &vec) const`
					`{`
					`  switch (active_EKF_type()) {`
					`#if AP_AHRS_DCM_ENABLED`
					`  case EKFType::DCM:`
					`  return dcm.get_relative_position_NED_origin(vec);`
					`#endif`
					`#if HAL_NAVEKF2_AVAILABLE`
					`  case EKFType::TWO: {`
					`  Vector2p posNE;`

5	AP_AHRS::get_velocity_NED()	state.velocity_NED	Copter::crash_check; AR_PosControl	Data dependency	`bool AP_AHRS::get_velocity_NED(Vector3f &vec) const`
					`{`
					`  vec = state.velocity_NED;`
					`  return state.velocity_NED_ok;`
					`}`

6	NavEKF3_core::UpdateFilter()	fill_scratch_variables(); update_sensor_selection(); Fuse*	AP_AHRS::update_EKF3() (AP_AHRS.cpp L692)	Function call	`void NavEKF3_core::UpdateFilter(bool predict)`
					`{`
					`  // don't run filter updates if states have not been initialised`
					`  if (!statesInitialised) {`
					`  return;`
					`  }`
					
					`  fill_scratch_variables();`
					
					`  // update sensor selection (for affinity)`

7	NavEKF3_core::SelectVelPosFusion()	readGpsData(); FuseVelPosNED()	NavEKF3_core::UpdateFilter()	Function call	`void NavEKF3_core::SelectVelPosFusion()`
					`{`
					`  // Check if the magnetometer has been fused on that time step and the filter is runnin`
					`g at faster than 200 Hz`
					`  // If so, don't fuse measurements on this time step to reduce frame over-runs`
					`  // Only allow one time slip to prevent high rate magnetometer data preventing fusion o`
					`f other measurements`
					`  if (magFusePerformed && dtIMUavg < 0.005f && !posVelFusionDelayed) {`
					`  posVelFusionDelayed = true;`
					`  return;`
					`  } else {`
					`  posVelFusionDelayed = false;`

8	AC_WPNav::set_wp_destination_loc()	get_vector_NEU_cm(); set_wp_destination_NEU_cm()	ArduCopter mode_auto / mode_guided	Function call	`bool AC_WPNav::set_wp_destination_loc(const Location& destination)`
					`{`
					`  bool is_terrain_alt;`
					`  Vector3f dest_neu_cm;`
					
					`  // convert destination location to vector`
					`  if (!get_vector_NEU_cm(destination, dest_neu_cm, is_terrain_alt)) {`
					`  return false;`
					`  }`

9	AP_Mission::get_next_nav_cmd()	get_next_cmd(); read_cmd_from_storage()	ArduCopter/Plane mission sequencing	Function call	`bool AP_Mission::get_next_nav_cmd(uint16_t start_index, Mission_Command& cmd)`
					`{`
					`  // search until the end of the mission command list`
					`  for (uint16_t cmd_index = start_index; cmd_index < (unsigned)_cmd_total; cmd_index++)`
					`{`
					`  // get next command`
					`  if (!get_next_cmd(cmd_index, cmd, false)) {`
					`  // no more commands so return failure`
					`  return false;`
					`  }`
					`  // if found a "navigation" command then return it`

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

10	AP_L1_Control::update_waypoint()	_ahrs.get_location(); AP_HAL::micros()	ArduPlane navigation update	Shared global / singleton	`void AP_L1_Control::update_waypoint(const Location &prev_WP, const Location &next_WP, floa t dist_min)`
					`{`
					`  Location _current_loc;`
					`  float Nu;`
					`  float xtrackVel;`
					`  float ltrackVel;`
					`  uint32_t now = AP_HAL::micros();`
					`  float dt = (now - _last_update_waypoint_us) * 1.0e-6f;`

11	AP_L1_Control::nav_roll_cd()	_ahrs.get_pitch_rad(); _latAccDem	Plane::calc_nav_roll() (Attitude.cpp L608)	Function call	`int32_t AP_L1_Control::nav_roll_cd(void) const`
					`{`
					`  float ret;`
					`  /*`
					`  formula can be obtained through equations of balanced spiral:`
					`  liftForce * cos(roll) = gravityForce * cos(pitch);`
					`  liftForce * sin(roll) = gravityForce * lateralAcceleration / gravityAcceleration;`
					`// as mass = gravityForce/gravityAcceleration`
					`  see issue 24319 [https://github.com/ArduPilot/ardupilot/issues/24319]`
					`  Multiplier 100.0f is for converting degrees to centidegrees`
					`  Made changes to avoid zero division as proposed by Andrew Tridgell: https://github .com/ArduPilot/ardupilot/pull/24331#discussion_r1267798397`

12	AP_TECS::update_pitch_throttle()	_update_pitch(); _update_throttle_with_airspeed()	ArduPlane stabilize/auto pitch loop	Function call	`void AP_TECS::update_pitch_throttle(int32_t hgt_dem_cm,`
					`  int32_t EAS_dem_cm,`
					`  enum AP_FixedWing::FlightStage flight_stage,`
					`  float distance_beyond_land_wp,`
					`  int32_t ptchMinCO_cd,`
					`  int16_t throttle_nudge,`
					`  float hgt_afe,`
					`  float load_factor,`
					`  float pitch_trim_deg)`
					`{`

13	AC_AttitudeControl::input_euler_angle _roll_pitch_yaw_cd()	input_euler_angle_roll_pitch_yaw_rad()	ArduCopter/ArduPlane flight-mode run()	Function call	`void AC_AttitudeControl::input_euler_angle_roll_pitch_yaw_cd(float euler_roll_angle_cd, fl oat euler_pitch_angle_cd, float euler_yaw_angle_cd, bool slew_yaw)`
					`{`
					`  // Convert from centidegrees on public interface to radians`
					`  const float euler_roll_angle_rad = cd_to_rad(euler_roll_angle_cd);`
					`  const float euler_pitch_angle_rad = cd_to_rad(euler_pitch_angle_cd);`
					`  const float euler_yaw_angle_rad = cd_to_rad(euler_yaw_angle_cd);`
					`  input_euler_angle_roll_pitch_yaw_rad(euler_roll_angle_rad, euler_pitch_angle_rad, eule r_yaw_angle_rad, slew_yaw);`
					`}`

Layer 2 — Full transitive chain to final consumer (hop count)

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
1	AP_GPS (via AP_AHRS)	**AP_GPS** → **AP_AHRS** → **ArduPlane::navigate()** → **ArduPlane::flight_mode_auto()**	ArduPlane::flight_mode_auto()	**3**	USER-SUPPLIED example chain (AAP 0.1.3/0.5.4); navigate() at ArduPlane/navigation.cpp:86 is gated on have_position.
2	AP_AHRS::groundspeed()	AP_GPS velocity → NavEKF3 → AP_AHRS::groundspeed() → Rover speed control → SRV_Channels	SRV_Channels	**4**	Ground-vehicle speed branch.
3	AP_AHRS::have_inertial_nav()	EKF health → AP_AHRS::have_inertial_nav() → Copter::ekf_has_absolute_position → position_ok() → set_mode gate	mode transition (allow/deny)	**4**	Also observed in Group 2 (Table 5 #7).
4	AP_AHRS::get_relative_position_NE D_origin()	EKF → AHRS NED → {AR_PosControl | AP_InertialNav} → actuators	SRV_Channels / AP_Motors	**3**	Geodetic↔NED transform branch.
5	AP_AHRS::get_velocity_NED()	EKF → AHRS → Copter::crash_check / controllers → disarm / AP_Motors	AP_Motors / disarm	**3**	Velocity consumers.
6	NavEKF3_core::UpdateFilter()	{AP_GPS, AP_InertialSensor, AP_Baro} → NavEKF3::UpdateFilter → AP_AHRS estimate → all consumers	AP_Motors	**4**	[CIRCULAR] AHRS↔EKF; EKF reads AP_DAL.
7	NavEKF3 PosVel fusion	AP_GPS readGpsData → FuseVelPosNED → EKF states → AP_AHRS	AP_AHRS	**3**	Core fusion transform.
8	AC_WPNav	AP_AHRS pos → AC_WPNav target → AC_PosControl → AC_AttitudeControl → AP_Motors	AP_Motors	**4**	Multirotor waypoint path.
9	AP_Mission	mission storage → get_next_nav_cmd → mode_auto → AC_WPNav → controllers	AP_Motors	**4**	Mission sequencing path.

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
10	AP_L1_Control::update_waypoint()	AP_AHRS pos → L1 guidance → nav_roll_cd() → Plane::calc_nav_roll → SRV_Channels	SRV_Channels	**4**	Fixed-wing lateral path.
11	AP_L1_Control::nav_roll_cd()	L1 latAccDem → nav_roll_cd() → Plane::calc_nav_roll → SRV_Channels	SRV_Channels	**3**	Roll-demand branch.
12	AP_TECS	airspeed/alt demand → TECS update_pitch_throttle → pitch/throttle → SRV_Channels	SRV_Channels	**3**	Energy/altitude branch.
13	AC_AttitudeControl (Sink)	nav demand → input_euler_* → attitude_controller_run_quat → rate_controller_run → AP_Motors	AP_Motors	**3**	Inbound chain to attitude Sink.

Table 3 — Core Timing (HAL clock primitives, RTC & time-source arbitration, jitter correction, scheduler, GPS time-of-week)

#	File Path	Line(s)	Function / Class	Role	Code Snippet (5-10 lines)	Notes

1	libraries/AP_HAL/system.h	15-20	AP_HAL::micros()/millis()/micros64()/millis64()	Source	`uint16_t micros16();`
					`uint32_t micros();`
					`uint32_t millis();`
					`uint16_t millis16();`
					`uint64_t micros64();`
					`uint64_t millis64();`
						Free-running monotonic clock primitives — the root timing Source. Backed by the ChibiOS RTOS tick (external boundary modules/ChibiOS). Consumed by every periodic loop and timestamp.

2	libraries/AP_HAL/Scheduler.h	15-21	AP_HAL::Scheduler::delay_microseconds()	Source	`    virtual void     delay(uint16_t ms) = 0;`
					``
					`    /*`
					`      delay for the given number of microseconds. This needs to be as`
					`      accurate as possible - preferably within 100 microseconds.`
					`     */`
					`    virtual void     delay_microseconds(uint16_t us) = 0;`
						Pure-virtual HAL scheduler timing interface; each board HAL backend (ChibiOS/Linux/SITL) overrides it.

3	libraries/AP_RTC/AP_RTC.h	32-40	AP_RTC::get_utc_usec() / set_utc_usec()	Source	`    /*`
					`      get clock in UTC microseconds.  Returns false if it is not available.`
					`     */`
					`    bool get_utc_usec(uint64_t &usec) const;`
					``
					`    // set the system time.  If the time has already been set by`
					`    // something better (according to source_type), this set will be`
					`    // ignored.`
					`    void set_utc_usec(uint64_t time_utc_usec, source_type type);`
						Real-time clock UTC accessor/mutator; reached via AP::rtc() singleton.

4	libraries/AP_RTC/AP_RTC.cpp	52-61	AP_RTC::set_utc_usec()	Transform	`void AP_RTC::set_utc_usec(uint64_t time_utc_usec, source_type type)`
					`{`
					`    const uint64_t oldest_acceptable_date_us = 1640995200ULL*1000*1000; // 2022-01-01 0:00`
					``
					`    // only allow time to be moved forward from the same sourcetype`
					`    // while the vehicle is disarmed:`
					`    if (hal.util->get_soft_armed()) {`
					`        if (type >= rtc_source_type) {`
					`            // e.g. system-time message when we've been set by the GPS`
						Arbitrates competing time sources by priority — a better/equal source is rejected while armed, so GPS time is not overwritten by a worse source.

5	libraries/AP_RTC/AP_RTC.h	22-30	AP_RTC::source_type	Source	`    // ordering is important in source_type; lower-numbered is`
					`    // considered a better time source.  These values are documented`
					`    // and used in the parameters!`
					`    enum source_type : uint8_t {`
					`        SOURCE_GPS = 0,`
					`        SOURCE_MAVLINK_SYSTEM_TIME = 1,`
					`        SOURCE_HW = 2,`
					`        SOURCE_NONE,`
					`    };`
						Clock-source priority enum (SOURCE_GPS=0 is best). Links GPS time-of-week (Positioning Source) into the Timing subsystem — cross-category coupling.

6	libraries/AP_RTC/JitterCorrection.cpp	50-59	JitterCorrection::correct_offboard_timestamp_usec()	Transform	`uint64_t JitterCorrection::correct_offboard_timestamp_usec(uint64_t offboard_usec, uint64_t local_usec)`
					`{`
					`    int64_t diff_us = int64_t(local_usec) - int64_t(offboard_usec);`
					``
					`    if (!initialised ||`
					`        diff_us < link_offset_usec) {`
					`        // this message arrived from the remote system with a`
					`        // timestamp that would imply the message was from the`
					`        // future. We know that isn't possible, so we adjust down the`
					`        // correction value`
						Time-sync transform: corrects offboard (GPS/MAVLink/CAN) timestamps to the local clock, compensating link jitter and lag.

7	libraries/AP_Scheduler/AP_Scheduler.h	149-155	AP_Scheduler::get_loop_period_us()	Transform	`    // get the time-allowed-per-loop in microseconds`
					`    uint32_t get_loop_period_us() {`
					`        if (_loop_period_us == 0) {`
					`            _loop_period_us = 1000000UL / _loop_rate_hz;`
					`        }`
					`        return _loop_period_us;`
					`    }`
						Derives the loop period from the configured rate: _loop_period_us = 1000000UL / _loop_rate_hz (L152).

8	libraries/AP_Scheduler/AP_Scheduler.cpp	347-356	AP_Scheduler::loop()	Sink	`void AP_Scheduler::loop()`
					`{`
					`    // wait for an INS sample`
					`    hal.util->persistent_data.scheduler_task = -3;`
					`    _rsem.give();`
					`    AP::ins().wait_for_sample();`
					`    _rsem.take_blocking();`
					`    hal.util->persistent_data.scheduler_task = -1;`
					``
					`    _loop_sample_time_us = AP_HAL::micros64();`
						Main scheduler loop; waits for the INS sample and stamps _loop_sample_time_us via AP_HAL::micros64(). Drives EKF prediction/update cadence (rate coupling).

#	File Path	Line(s)	Function / Class	Role	Code Snippet (5-10 lines)	Notes

9	libraries/AP_Scheduler/AP_Scheduler.cpp	168-172	AP_Scheduler::tick()	Transform	`void AP_Scheduler::tick(void)`
					`{`
					`  _tick_counter++;`
					`  _tick_counter32++;`
					`}`
						Increments the tick counter that the main-loop-lockup watchdog samples (see Table 6 #1).

10	libraries/AP_GPS/AP_GPS.h	409-418	AP_GPS::time_week_ms() / time_week()	Source	`// GPS time of week in milliseconds`
					`uint32_t time_week_ms(uint8_t instance) const {`
					`  return state[instance].time_week_ms;`
					`}`
					`uint32_t time_week_ms() const {`
					`  return time_week_ms(primary_instance);`
					`}`
					
					`// GPS week`
					`uint16_t time_week(uint8_t instance) const {`
						[MULTI-CATEGORY] GPS time-of-week accessors — same GPS_State as Table 1 #5 (position). GPS time feeds AP_RTC (SOURCE_GPS) and EKF time alignment.

11	libraries/AP_GPS/AP_GPS.h	196-201	AP_GPS::GPS_State (time_week_ms / time_week fields)	Source	`GPS_Status status;  ///< driver fix status`
					`uint32_t time_week_ms;  ///< GPS time (milliseconds from start of GPS week)`
					`uint16_t time_week;  ///< GPS week number`
					`Location location;  ///< last fix location`
					`float ground_speed;  ///< ground speed in m/s`
					`float ground_course;  ///< ground course in degrees, wrapped 0-360`
						[SHARED-STRUCT][MULTI-CATEGORY] Timing fields time_week_ms/time_week packed in GPS_State alongside Positioning (location) and Navigation (velocity/ground_speed) — see Table 1 #5.

Table 3a — Timing dependencies (Layer 1 direct edges + Layer 2 transitive chains)

Layer 1 — Direct calls / called-by with typed dependency

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

1	AP_HAL::micros()/millis()/micros64()	ChibiOS RTOS tick / hw timer	AP_Scheduler::loop (micros64 L356); Copter::failsafe_check (micros L37); pervasive	Function call	`uint16_t micros16();`
					`uint32_t micros();`
					`uint32_t millis();`
					`uint16_t millis16();`
					`uint64_t micros64();`
					`uint64_t millis64();`

2	AP_HAL::Scheduler::delay_microseconds()	(pure virtual)	HAL backend override (ChibiOS/Linux/SITL)	Inheritance / Interface	`virtual void  delay(uint16_t ms) = 0;`
					
					`/*`
					`  delay for the given number of microseconds. This needs to be as`
					`  accurate as possible - preferably within 100 microseconds.`
					`*/`
					`virtual void  delay_microseconds(uint16_t us) = 0;`

3	AP_RTC::get_utc_usec()	rtc_shift + AP_HAL::micros64()	Logging, MAVLink SYSTEM_TIME, scripting	Shared global / singleton	`/*`
					`  get clock in UTC microseconds.  Returns false if it is not available.`
					`*/`
					`bool get_utc_usec(uint64_t &usec) const;`
					
					`// set the system time.  If the time has already been set by`
					`// something better (according to source_type), this set will be`
					`// ignored.`
					`void set_utc_usec(uint64_t time_utc_usec, source_type type);`

4	AP_RTC::set_utc_usec()	compares type vs rtc_source_type	AP_GPS (GPS time); GCS SYSTEM_TIME handler	Function call	`void AP_RTC::set_utc_usec(uint64_t time_utc_usec, source_type type)`
					`{`
					`  const uint64_t oldest_acceptable_date_us = 1640995200ULL*1000*1000; // 2022-01-01 0:00`
					
					`  // only allow time to be moved forward from the same sourcetype`
					`  // while the vehicle is disarmed:`
					`  if (hal.util->get_soft_armed()) {`
					`  if (type >= rtc_source_type) {`
					`  // e.g. system-time message when we've been set by the GPS`
					`  return;`

5	AP_RTC::source_type (enum)	(enum values)	AP_RTC::set_utc_usec() priority arbitration	Data dependency	`// ordering is important in source_type; lower-numbered is`
					`// considered a better time source.  These values are documented`
					`// and used in the parameters!`
					`enum source_type : uint8_t {`
					`  SOURCE_GPS = 0,`
					`  SOURCE_MAVLINK_SYSTEM_TIME = 1,`
					`  SOURCE_HW = 2,`
					`  SOURCE_NONE,`
					`};`

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

6	JitterCorrection::correct_offboard_timestamp_usec()	int64 arithmetic on local vs offboard us	AP_GPS (DroneCAN/MAV), AP_VisualOdom, MAVLink ts	Function call	`uint64_t JitterCorrection::correct_offboard_timestamp_usec(uint64_t offboard_usec, uint64_t local_usec)`
					`{`
					`  int64_t diff_us = int64_t(local_usec) - int64_t(offboard_usec);`
					
					`  if (!initialised ||`
					`  diff_us < link_offset_usec) {`
					`  // this message arrived from the remote system with a`
					`  // timestamp that would imply the message was from the`
					`  // future. We know that isn't possible, so we adjust down the`
					`  // correction value`

7	AP_Scheduler::get_loop_period_us()	_loop_rate_hz	AP_Scheduler::run() budget; vehicle fast loop	Function call	`  // get the time-allowed-per-loop in microseconds`
					`  uint32_t get_loop_period_us() {`
					`  if (_loop_period_us == 0) {`
					`  _loop_period_us = 1000000UL / _loop_rate_hz;`
					`  }`
					`  return _loop_period_us;`
					`  }`

8	AP_Scheduler::loop()	AP::ins().wait_for_sample(); AP_HAL::micros64(); run()	Vehicle main loop (Copter/Plane/Rover/Sub)	Shared global / singleton	`void AP_Scheduler::loop()`
					`{`
					`  // wait for an INS sample`
					`  hal.util->persistent_data.scheduler_task = -3;`
					`  _rsem.give();`
					`  AP::ins().wait_for_sample();`
					`  _rsem.take_blocking();`
					`  hal.util->persistent_data.scheduler_task = -1;`
					
					`  _loop_sample_time_us = AP_HAL::micros64();`

9	AP_Scheduler::tick()	_tick_counter++ / _tick_counter32++	AP_Scheduler::loop()	Data dependency	`void AP_Scheduler::tick(void)`
					`{`
					`  _tick_counter++;`
					`  _tick_counter32++;`
					`}`

10	AP_GPS::time_week_ms()/time_week()	state[instance].time_week_ms / time_week	AP_RTC (GPS time source); EKF time alignment	Shared global / singleton	`  // GPS time of week in milliseconds`
					`  uint32_t time_week_ms(uint8_t instance) const {`
					`  return state[instance].time_week_ms;`
					`  }`
					`  uint32_t time_week_ms() const {`
					`  return time_week_ms(primary_instance);`
					`  }`
					
					`  // GPS week`
					`  uint16_t time_week(uint8_t instance) const {`

11	GPS_State.time_week_ms / time_week (fields)	(data fields)	GPS backends (write); time accessors (read)	Data dependency	`  GPS_Status status;  ///< driver fix status`
					`  uint32_t time_week_ms;  ///< GPS time (milliseconds from start of GPS week)`
					`  uint16_t time_week;  ///< GPS week number`
					`  Location location;  ///< last fix location`
					`  float ground_speed;  ///< ground speed in m/s`
					`  float ground_course;  ///< ground course in degrees, wrapped 0-360`

Layer 2 — Full transitive chain to final consumer (hop count)

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
1	AP_HAL::micros()/64	AP_HAL clock → AP_Scheduler loop/tick → {NavEKF3::UpdateFilter, AC_WPNav, AC_AttitudeControl}	periodic PNT tasks → AP_Motors	**3**	Timing spine root drives all PNT updates.
2	Scheduler::delay_microseconds	HAL clock → scheduler delay → loop pacing	loop cadence	**2**	Loop timing pacing.
3	AP_RTC::get_utc_usec()	RTOS+GPS time → AP_RTC → log timestamps / MAVLink	AP_Logger / GCS	**3**	UTC reporting branch.
4	AP_RTC::set_utc_usec()	AP_GPS time_week → AP_RTC::set_utc_usec → system UTC clock	AP_RTC clock	**2**	GPS time → RTC (SOURCE_GPS).
5	AP_RTC::source_type	{GPS, MAVLink, HW} → source_type priority → accepted RTC time	AP_RTC	**2**	Source arbitration.
6	JitterCorrection	offboard timestamp → correct_offboard_timestamp_usec → aligned local time → EKF/GPS fusion	NavEKF3	**3**	Timestamp alignment for fusion.
7	get_loop_period_us()	_loop_rate_hz → loop period → task time budget → all scheduled tasks	scheduler tasks	**2**	Rate → period derivation.
8	AP_Scheduler::loop()	micros64 + INS sample → loop() → run() → {EKF, NAV, ATT}	AP_Motors	**3**	Drives every periodic PNT update.

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
9	AP_Scheduler::tick()	tick++ → Copter::failsafe_check watchdog → motors output_min	AP_Motors / disarm	**2**	Feeds Table 6 #1 timing watchdog.
10	AP_GPS time-of-week	GPS_State → time_week_ms()/time_week() → AP_RTC / EKF time align	AP_RTC / NavEKF3	**2**	[MULTI-CATEGORY] also Table 1.
11	GPS_State.time_week (fields)	GPS backend write → GPS_State.time_week → accessors → AP_RTC	AP_RTC	**2**	[SHARED-STRUCT][MULTI-CATEGORY].

GROUP 2 — INDIRECT / RELATIONAL PNT REFERENCES

INDIRECT / RELATIONAL — not core. This code does not directly read or write PNT data; its behavior *changes as a result of* PNT state. Each row records the **Trigger Condition**, the **PNT State Observed** (the discriminator from Group 1), and the **Behavior Change Description**.

Table 4 — Indirect / Relational — Positioning (mode position-gating, geofence, GPS-denied fallback)

#	File Path	Line(s)	Function / Class	Trigger Condition	PNT State Observed	Behavior Change Description	Code Snippet (5-10 lines)	Notes
1	ArduCopter/mode.cpp	304-309	Copter::set_mode()	Pilot/auto/failsafe requests a flight-mode change	new_flightmode->requires_GPS() && !copter.position_ok()	Mode change is DENIED with “requires position” when the mode needs GPS but no position estimate exists.	`if (!ignore_checks && new_flightmode->requires_GPS() && !copter.position_ok()) { mode_change_failed(new_flightmode, "requires position"); return false; }`	Indirect positioning: gates mode entry on position health. position_ok() catalogued at Table 5 #2 / Table 4 #2.
2	ArduCopter/system.cpp	226-234	Copter::position_ok()	Any consumer queries vehicle position health	ekf_has_absolute_position() || ekf_has_relative_position(); failsafe.ekf	Returns false (blocking position-dependent modes/features) if EKF failsafe is active or no position estimate.	`bool Copter::position_ok() const { // return false if ekf failsafe has triggered if (failsafe.ekf) { return false; } // check ekf position estimate return (ekf_has_absolute_position() || ekf_has_relative_position()); }`	Central position-health predicate; observes EKF position state without reading raw PNT values.
3	ArduCopter/fence.cpp	8-17	Copter::fence_checks_async()	Async IO tick (25 Hz gate via AP_HAL::timeout_expired 40 ms)	fence.get_breaches(); fence.check() — vehicle position vs geofence boundary	Computes new fence breaches that drive the fence failsafe action (RTL/LAND/etc.).	`void Copter::fence_checks_async() { const uint32_t now = AP_HAL::millis(); if (!AP_HAL::timeout_expired(fence_breaches.last_check_ms, now, 40U)) { // 25Hz update rate return; } if (fence_breaches.have_updates) { return; // wait for the main loop to pick up the new breaches before checking again }`	Positioning-indirect (geofence). The 40 ms timing gate also appears in Table 6 #4 [MULTI-CATEGORY trigger].
4	libraries/AC_Fence/AC_Fence.cpp	753-762	AC_Fence::check()	Periodic fence evaluation	Vehicle position vs polygon / circle / altitude fence	Returns a breach bitmask consumed by the vehicle fence handler to initiate failsafe.	`uint8_t AC_Fence::check(bool disable_auto_fences) { uint8_t ret = 0; uint8_t disabled_fences = disable_auto_fences ? get_auto_disable_fences() : 0; uint8_t fences_to_disable = disabled_fences & _enabled_fences; // clear any breach from a non-enabled fence clear_breach(~_configured_fences); // clear any breach from disabled fences clear_breach(fences_to_disable);`	Supporting library; reacts to position relative to configured fences.
5	ArduCopter/mode_guided_nogps.cpp	10-15	ModeGuidedNoGPS::init()	Mode entry request for GUIDED_NOGPS	(none — explicitly GPS-independent)	Allows guided angle control WITHOUT any position estimate; bypasses the position gate of row 1.	`bool ModeGuidedNoGPS::init(bool ignore_checks) { // start in angle control mode ModeGuided::angle_control_start(); return true; }`	[ABSENT] position gate intentionally omitted — the GPS-denied fallback contrasted with position-gated modes.
6	ArduPlane/takeoff.cpp	58-64	Plane::auto_takeoff_check()	Automatic takeoff initiation	gps.status() < AP_GPS::GPS_OK_FIX_3D	Blocks automatic takeoff when there is no 3D GPS lock.	`// Check for bad GPS if (gps.status() < AP_GPS::GPS_OK_FIX_3D) { // no auto takeoff without GPS lock return false; } bool do_takeoff_attitude_check = !(flight_option_enabled(FlightOptions::DISABLE_TOFF_ATTITUDE_CHK));`	Cites user example gps.status() (= AP_GPS::status(), defined at libraries/AP_GPS/AP_GPS.h L293).

#	File Path	Line(s)	Function / Class	Trigger Condition	PNT State Observed	Behavior Change Description	Code Snippet (5-10 lines)	Notes

7	Rover/mode.h	84-90	Rover::Mode::requires_position()	Drive-mode entry / mode capability query	requires_position() / requires_velocity() (per-mode virtual)	Rover mode entry is gated on a valid position/velocity estimate (overridden false in MANUAL/HOLD/etc.).	`// true if heading is controlled`
							`virtual bool attitude_stabilized() const { return true; }`
							`// true if mode requires position and/or velocity estimate`
							`virtual bool requires_position() const { return true; }`
							`virtual bool requires_velocity() const { return true; }`
								Indirect positioning gating across Rover drive modes (14 mode files).

Table 4a — Indirect Positioning dependencies (Layer 1 + Layer 2)

Layer 1 — Direct calls / called-by with typed dependency

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet
1	Copter::set_mode()	requires_GPS(); position_ok(); mode_change_failed()	GCS/RC mode switch; failsafe handlers	Function call	`if (!ignore_checks && new_flightmode->requires_GPS() && !copter.position_ok()) { mode_change_failed(new_flightmode, "requires position"); return false; }`
2	Copter::position_ok()	ekf_has_absolute_position(); ekf_has_relative_position()	Copter::set_mode(); arming; mode run()	Function call	`bool Copter::position_ok() const { // return false if ekf failsafe has triggered if (failsafe.ekf) { return false; } // check ekf position estimate return (ekf_has_absolute_position() || ekf_has_relative_position()); }`
3	Copter::fence_checks_async()	AP_HAL::timeout_expired(); fence.check(); fence.get_breaches()	IO timer (1 kHz async callback)	Function call	`void Copter::fence_checks_async() { const uint32_t now = AP_HAL::millis(); if (!AP_HAL::timeout_expired(fence_breaches.last_check_ms, now, 40U)) { // 25Hz update rate return; } if (fence_breaches.have_updates) { return; // wait for the main loop to pick up the new breaches before checking again }`
4	AC_Fence::check()	get_auto_disable_fences(); clear_breach(); polygon/circle/alt checks	Copter::fence_checks_async()	Function call	`uint8_t AC_Fence::check(bool disable_auto_fences) { uint8_t ret = 0; uint8_t disabled_fences = disable_auto_fences ? get_auto_disable_fences() : 0; uint8_t fences_to_disable = disabled_fences & _enabled_fences; // clear any breach from a non-enabled fence clear_breach(~_configured_fences); // clear any breach from disabled fences clear_breach(fences_to_disable);`
5	ModeGuidedNoGPS::init()	ModeGuided::angle_control_start()	Copter::set_mode(GUIDED_NOGPS)	Inheritance / Interface	`bool ModeGuidedNoGPS::init(bool ignore_checks) { // start in angle control mode ModeGuided::angle_control_start(); return true; }`
6	Plane::auto_takeoff_check()	gps.status(); flight_option_enabled()	ModeAuto / ModeTakeoff sequencing	Shared global / singleton	`// Check for bad GPS if (gps.status() < AP_GPS::GPS_OK_FIX_3D) { // no auto takeoff without GPS lock return false; } bool do_takeoff_attitude_check = !(flight_option_enabled(FlightOptions::DISABLE_TOFF_ATTITUDE_CHK));`

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

7	Rover::Mode::requires_position()	(virtual predicate)	Rover::Mode::enter() entry gate	Inheritance / Interface	`// true if heading is controlled`
					`virtual bool attitude_stabilized() const { return true; }`
					
					`// true if mode requires position and/or velocity estimate`
					`virtual bool requires_position() const { return true; }`
					`virtual bool requires_velocity() const { return true; }`

Layer 2 — Full transitive chain to final consumer (hop count)

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
1	Copter::set_mode (position gate)	AP_GPS/EKF position → position_ok() → set_mode() gate → mode_change_failed()	denied mode transition	**3**	Indirect positioning gate.
2	Copter::position_ok()	EKF position estimate → ekf_has_*_position() → position_ok() → mode/arming gates	mode availability	**3**	Position-health fan-out.
3	Copter::fence_checks_async()	vehicle position → AC_Fence::check() → fence_breaches → fence failsafe (RTL/LAND)	mode transition	**3**	Geofence reaction.
4	AC_Fence::check()	position vs fence → breach bitmask → Copter fence handler → failsafe action	mode transition	**3**	Breach computation.
5	ModeGuidedNoGPS::init()	(no PNT) → angle_control_start() → AC_AttitudeControl → AP_Motors	AP_Motors	**2**	[ABSENT] no position gate (GPS-denied).
6	Plane::auto_takeoff_check()	AP_GPS::status() → auto_takeoff_check() gate → takeoff allowed/blocked	takeoff decision	**2**	Observes gps.status().
7	Rover requires_position()	EKF position → requires_position() → Rover Mode::enter() gate	drive-mode entry	**2**	Rover mode gating.

Table 5 — Indirect / Relational — Navigation (EKF-health watchdog, nav-output reporting & consumption, crash/flight detection)

#	File Path	Line(s)	Function / Class	Trigger Condition	PNT State Observed	Behavior Change Description	Code Snippet (5-10 lines)	Notes

1	ArduCopter/ekf_check.cpp	30-39	Copter::ekf_check()	Runs at 10 Hz; EKF variance over threshold for EKF_CHECK_ITERATIONS_MAX (=10) iterations	ekf_over_threshold(); ekf_has_relative_position()/ekf_has_absolute_position(); ahrs.get_origin()	Trips the EKF failsafe (failsafe_ekf_event) which forces a mode change / land.	`void Copter::ekf_check()`
							`{`
							`  // ensure EKF_CHECK_ITERATIONS_MAX is at least 7`
							`  static_assert(EKF_CHECK_ITERATIONS_MAX >= 7, "EKF_CHECK_ITERATIONS_MAX must be at least 7");`
							`  // exit immediately if ekf has no origin yet - this assumes the origin can never become unset`
							`  Location temp_loc;`
							`  if (!ahrs.get_origin(temp_loc)) {`
							`  return;`
							`  }`
								[DUPLICATED] near-identical watchdog to ArduPlane/ekf_check.cpp (row 2) — extraction risk.

2	ArduPlane/ekf_check.cpp	30-39	Plane::ekf_check()	Runs at 10 Hz; EKF variance over threshold for EKF_CHECK_ITERATIONS_MAX (=10) iterations	ekf_over_threshold(); ahrs.get_origin()	Trips the EKF failsafe on Plane.	`void Plane::ekf_check()`
							`{`
							`  // ensure EKF_CHECK_ITERATIONS_MAX is at least 7`
							`  static_assert(EKF_CHECK_ITERATIONS_MAX >= 7, "EKF_CHECK_ITERATIONS_MAX must be at least 7");`
							`  // exit immediately if ekf has no origin yet - this assumes the origin can never become unset`
							`  Location temp_loc;`
							`  if (!ahrs.get_origin(temp_loc)) {`
							`  return;`
							`  }`
								[DUPLICATED] copy of Copter logic: same EKF_CHECK_ITERATIONS_MAX=10, ekf_check_state struct, static_assert>=7.

3	ArduPlane/GCS_MAVLink_Plane.cpp	233-242	GCS_MAVLINK_Plane::send_nav_controller_output()	GCS telemetry stream request	nav_controller->nav_bearing_cd(); target_bearing_cd(); crosstrack_error()	Emits the NAV_CONTROLLER_OUTPUT MAVLink message reporting navigation health/targets to the GCS.	`  mavlink_msg_nav_controller_output_send(`
							`  chan,`
							`  plane.nav_roll_cd * 0.01,`
							`  plane.nav_pitch_cd * 0.01,`
							`  nav_controller->nav_bearing_cd() * 0.01,`
							`  nav_controller->target_bearing_cd() * 0.01,`
							`  MIN(plane.auto_state.wp_distance, UINT16_MAX),`
							`  plane.calc_altitude_error_cm() * 0.01,`
							`  plane.airspeed_error * 100,  // incorrect units; see PR#7933`
							`  nav_controller->crosstrack_error());`
								External boundary modules/mavlink (message definitions).

4	ArduPlane/Attitude.cpp	606-611	Plane::calc_nav_roll()	Attitude control loop	nav_controller->nav_roll_cd()	Sets the commanded roll (nav_roll_cd) from the navigation controller output (Sink of nav state).	`void Plane::calc_nav_roll()`
							`{`
							`  int32_t commanded_roll = nav_controller->nav_roll_cd();`
							`  nav_roll_cd = constrain_int32(commanded_roll, -roll_limit_cd, roll_limit_cd);`
							`  update_load_factor();`
							`}`
								Consumes navigation output without reading raw PNT.

5	ArduPlane/navigation.cpp	86-95	Plane::navigate()	Navigation update tick	have_position (position-estimate validity)	Skips navigation entirely when no valid position is available.	`void Plane::navigate()`
							`{`
							`  // do not navigate with corrupt data`
							`  // ---------------------------------`
							`  if (!have_position) {`
							`  return;`
							`  }`
							`  if (next_WP_loc.lat == 0 && next_WP_loc.lng == 0) {`
							`  return;`
								Explicit hop in the user-supplied chain (Table 2a #1).

6	ArduCopter/crash_check.cpp	64-73	Copter::crash_check()	Armed and not landed; per scheduler tick	ahrs.cos_roll()/cos_pitch() (attitude); ahrs.get_velocity_NED(); attitude error	Disarms the vehicle when a crash is detected from large angle error at speed.	`  // check for lean angle over 15 degrees`
							`  const float lean_angle_deg = degrees(acosf(ahrs.cos_roll()*ahrs.cos_pitch()));`
							`  if (lean_angle_deg <= CRASH_CHECK_ANGLE_MIN_DEG) {`
							`  crash_counter = 0;`
							`  return;`
							`  }`
							`  // check for angle error over 30 degrees`
							`  const float angle_error = attitude_control->get_att_error_angle_deg();`
								Reacts to navigation/attitude state; does not write PNT.

#	File Path	Line(s)	Function / Class	Trigger Condition	PNT State Observed	Behavior Change Description	Code Snippet (5-10 lines)	Notes
7	ArduCopter/system.cpp	238-246	Copter::ekf_has_absolute_position()	Position-health query	ahrs.have_inertial_nav()	Denies an absolute position estimate when only DCM (no inertial nav) is available.	`bool Copter::ekf_has_absolute_position() const { if (!ahrs.have_inertial_nav()) { // do not allow navigation with dcm position return false; } // if disarmed we accept a predicted horizontal position if (!motors->armed()) {`	Cites user example ahrs.have_inertial_nav() (defined libraries/AP_AHRS/AP_AHRS.h L273).
8	ArduPlane/is_flying.cpp	23-31	Plane::update_is_flying_5Hz()	Flight-state detection at 5 Hz	gps.status() >= AP_GPS::GPS_OK_FIX_3D; gps.ground_speed_cm()	Sets gps_confirmed_movement, influencing the is-flying probability and landing/failsafe logic.	`uint32_t ground_speed_thresh_cm = (aparm.min_groundspeed > 0) ? ((uint32_t)(aparm.min_groundspeed*(100*0.9))) : GPS_IS_FLYING_SPEED_CMS; bool gps_confirmed_movement = (gps.status() >= AP_GPS::GPS_OK_FIX_3D) && (gps.ground_speed_cm() >= ground_speed_thresh_cm); // airspeed at least 75% of stall speed? const float airspeed_threshold = MAX(aparm.airspeed_min,2)*0.75f; bool airspeed_movement = ahrs.airspeed_estimate(aspeed) && (aspeed >= airspeed_threshold); if (gps.status() < AP_GPS::GPS_OK_FIX_2D && arming.is_armed() && !airspeed_movement && isFlyingProbability > 0.3) {`	Cites user example gps.status().
9	ArduSub/failsafe.cpp	111-120	Sub::failsafe_ekf_check()	EKF compass & velocity variance exceed fs_ekf_thresh (latched after 2 s)	ahrs.get_variances(vel_variance, posVar, hgtVar, magVar, tasVar); compass_variance = magVar.length()	Trips the EKF failsafe (failsafe.ekf = true) on the submarine when the navigation solution degrades.	`float posVar, hgtVar, tasVar; Vector3f magVar; float compass_variance; float vel_variance; ahrs.get_variances(vel_variance, posVar, hgtVar, magVar, tasVar); compass_variance = magVar.length(); if (compass_variance < g.fs_ekf_thresh && vel_variance < g.fs_ekf_thresh) { last_ekf_good_ms = AP_HAL::millis(); failsafe.ekf = false;`	[DUPLICATED] third copy of the EKF-variance watchdog (cf. ArduCopter/ekf_check.cpp #1 and ArduPlane/ekf_check.cpp #2) — extraction risk across three vehicles.

Table 5a — Indirect Navigation dependencies (Layer 1 + Layer 2)

Layer 1 — Direct calls / called-by with typed dependency

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet
1	Copter::ekf_check()	ekf_over_threshold(); ekf_has_*_position(); ahrs.get_origin(); failsafe_ekf_event()	Scheduler 10 Hz task table	Function call	`void Copter::ekf_check() { // ensure EKF_CHECK_ITERATIONS_MAX is at least 7 static_assert(EKF_CHECK_ITERATIONS_MAX >= 7, "EKF_CHECK_ITERATIONS_MAX must be at least 7"); // exit immediately if ekf has no origin yet - this assumes the origin can never become unset Location temp_loc; if (!ahrs.get_origin(temp_loc)) { return; }`
2	Plane::ekf_check()	ekf_over_threshold(); ahrs.get_origin(); failsafe_ekf_*()	Scheduler 10 Hz task table	Function call	`void Plane::ekf_check() { // ensure EKF_CHECK_ITERATIONS_MAX is at least 7 static_assert(EKF_CHECK_ITERATIONS_MAX >= 7, "EKF_CHECK_ITERATIONS_MAX must be at least 7"); // exit immediately if ekf has no origin yet - this assumes the origin can never become unset Location temp_loc; if (!ahrs.get_origin(temp_loc)) { return; }`

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

3	send_nav_controller_output()	nav_controller->nav_bearing_cd()/target_bearing_cd()/crosstrack_error()	GCS_MAVLINK message streaming	Function call	`mavlink_msg_nav_controller_output_send(`
					`chan,`
					`plane.nav_roll_cd * 0.01,`
					`plane.nav_pitch_cd * 0.01,`
					`nav_controller->nav_bearing_cd() * 0.01,`
					`nav_controller->target_bearing_cd() * 0.01,`
					`MIN(plane.auto_state.wp_distance, UINT16_MAX),`
					`plane.calc_altitude_error_cm() * 0.01,`
					`plane.airspeed_error * 100, // incorrect units; see PR#7933`
					`nav_controller->crosstrack_error());`

4	Plane::calc_nav_roll()	nav_controller->nav_roll_cd(); update_load_factor()	ArduPlane stabilize/auto attitude loop	Function call	`void Plane::calc_nav_roll()`
					`{`
					`int32_t commanded_roll = nav_controller->nav_roll_cd();`
					`nav_roll_cd = constrain_int32(commanded_roll, -roll_limit_cd, roll_limit_cd);`
					`update_load_factor();`
					`}`

5	Plane::navigate()	check_home_alt_change(); update_waypoint(); update_loiter()	ArduPlane fast loop (gated on have_position)	Data dependency	`void Plane::navigate()`
					`{`
					`// do not navigate with corrupt data`
					`// ---------------------------------`
					`if (!have_position) {`
					`return;`
					`}`
					
					`if (next_WP_loc.lat == 0 && next_WP_loc.lng == 0) {`
					`return;`

6	Copter::crash_check()	ahrs.cos_roll()/cos_pitch(); ahrs.get_velocity_NED(); get_att_error_angle_deg()	Scheduler 10 Hz task	Function call	`// check for lean angle over 15 degrees`
					`const float lean_angle_deg = degrees(acosf(ahrs.cos_roll()*ahrs.cos_pitch()));`
					`if (lean_angle_deg <= CRASH_CHECK_ANGLE_MIN_DEG) {`
					`crash_counter = 0;`
					`return;`
					`}`
					
					`// check for angle error over 30 degrees`
					`const float angle_error = attitude_control->get_att_error_angle_deg();`

7	Copter::ekf_has_absolute_position()	ahrs.have_inertial_nav()	Copter::position_ok()	Function call	`bool Copter::ekf_has_absolute_position() const`
					`{`
					`if (!ahrs.have_inertial_nav()) {`
					`// do not allow navigation with dcm position`
					`return false;`
					`}`
					
					`// if disarmed we accept a predicted horizontal position`
					`if (!motors->armed()) {`

8	Plane::update_is_flying_5Hz()	gps.status(); gps.ground_speed_cm(); ahrs.airspeed_estimate()	Scheduler 5 Hz task	Shared global / singleton	`uint32_t ground_speed_thresh_cm = (aparm.min_groundspeed > 0) ? ((uint32_t)(aparm.min_groundspeed*(100*0.9))) : GPS_IS_FLYING_SPEED_CMS;`
					`bool gps_confirmed_movement = (gps.status() >= AP_GPS::GPS_OK_FIX_3D) &&`
					`(gps.ground_speed_cm() >= ground_speed_thresh_cm);`
					
					`// airspeed at least 75% of stall speed?`
					`const float airspeed_threshold = MAX(aparm.airspeed_min,2)*0.75f;`
					`bool airspeed_movement = ahrs.airspeed_estimate(aspeed) && (aspeed >= airspeed_threshold);`
					
					`if (gps.status() < AP_GPS::GPS_OK_FIX_2D && arming.is_armed() && !airspeed_movement && isFlyingProbability > 0.3) {`

9	Sub::failsafe_ekf_check()	ahrs.get_variances(); AP_HAL::millis(); gcs().send_text()	Scheduler periodic task (ArduSub)	Shared global / singleton	`float posVar, hgtVar, tasVar;`
					`Vector3f magVar;`
					`float compass_variance;`
					`float vel_variance;`
					`ahrs.get_variances(vel_variance, posVar, hgtVar, magVar, tasVar);`
					`compass_variance = magVar.length();`
					
					`if (compass_variance < g.fs_ekf_thresh && vel_variance < g.fs_ekf_thresh) {`
					`last_ekf_good_ms = AP_HAL::millis();`
					`failsafe.ekf = false;`

Layer 2 — Full transitive chain to final consumer (hop count)

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
1	Copter::ekf_check()	EKF variance/health → ekf_check() → failsafe_ekf_event() → set_mode(LAND/RTL)	mode transition	**3**	[DUPLICATED] vs Plane.
2	Plane::ekf_check()	EKF variance → ekf_check() → failsafe_ekf_*() → mode/AFS	mode transition	**3**	[DUPLICATED] vs Copter.
3	send_nav_controller_output()	nav_controller state → send_nav_controller_output() → MAVLink NAV_CONTROLLER_OUTPUT → GCS	GCS operator	**3**	modules/mavlink boundary.
4	Plane::calc_nav_roll()	L1/nav_controller → nav_roll_cd() → calc_nav_roll() → SRV_Channels	SRV_Channels	**3**	Nav-output sink.
5	Plane::navigate()	AHRS position (have_position) → navigate() → update_waypoint/loiter → L1 → SRV_Channels	SRV_Channels	**4**	User 2a chain hop.
6	Copter::crash_check()	AHRS attitude + EKF velocity → crash_check() → disarm	disarm / AP_Motors	**2**	Crash reaction.
7	Copter::ekf_has_absolute_position()	ahrs.have_inertial_nav() → ekf_has_absolute_position() → position_ok() → mode gate	mode availability	**3**	Observes have_inertial_nav().
8	Plane::update_is_flying_5Hz()	gps.status() → update_is_flying_5Hz() → is_flying state → failsafe/landing logic	flight-state machine	**2**	Observes gps.status().
9	Sub::failsafe_ekf_check()	EKF variances (ahrs.get_variances) → failsafe_ekf_check() → failsafe.ekf → mode/surface action	failsafe state (ArduSub)	**3**	[DUPLICATED] watchdog across Copter/Plane/Sub.

Table 6 — Indirect / Relational — Timing (main-loop-lockup watchdogs, PNT-health message hysteresis, rate gating)

#	File Path	Line(s)	Function / Class	Trigger Condition	PNT State Observed	Behavior Change Description	Code Snippet (5-10 lines)	Notes

1	ArduCopter/failsafe.cpp	35-44	Copter::failsafe_check()	1 kHz timer ISR; main loop has not advanced scheduler.ticks() (lockup) for > 2,000,000 us	AP_HAL::micros(); scheduler.ticks()	Declares CPU/main-loop failsafe and commands motors->output_min() when the loop stalls > 2 s.	`void Copter::failsafe_check()`
							`{`
							`  uint32_t tnow = AP_HAL::micros();`
							
							`  const uint16_t ticks = scheduler.ticks();`
							`  if (ticks != failsafe_last_ticks) {`
							`  // the main loop is running, all is OK`
							`  failsafe_last_ticks = ticks;`
							`  failsafe_last_timestamp = tnow;`
							`  if (in_failsafe) {`
								[DUPLICATED] timing watchdog mirrored in ArduPlane/failsafe.cpp (row 2).

2	ArduPlane/failsafe.cpp	17-26	Plane::failsafe_check()	Timer ISR; main loop stalled (ticks unchanged)	micros(); scheduler.ticks()	Enters failsafe / advanced-failsafe path when the loop stalls (200 ms threshold, 20 ms recovery).	`void Plane::failsafe_check(void)`
							`{`
							`  static uint16_t last_ticks;`
							`  static uint32_t last_timestamp;`
							`  static bool in_failsafe;`
							`  uint32_t tnow = micros();`
							
							`  const uint16_t ticks = scheduler.ticks();`
							`  if (ticks != last_ticks) {`
							`  // the main loop is running, all is OK`
								[DUPLICATED] copy of Copter timing watchdog with different thresholds.

3	ArduCopter/ekf_check.cpp	83-90	Copter::ekf_check() (GCS warning throttle)	EKF bad-variance declared	AP_HAL::millis() - last_warn_time > EKF_CHECK_WARNING_TIME (30 s)	Rate-limits the “EKF variance” GCS warning to at most once per 30 s (timing hysteresis).	`LOGGER_WRITE_ERROR(LogErrorSubsystem::EKFCHECK, LogErrorCode::EKFCHECK_BAD_VARIANCE);`
							`// send message to gcs`
							`if ((AP_HAL::millis() - ekf_check_state.last_warn_time) > EKF_CHECK_WARNING_TIME) {`
							`gcs().send_text(MAV_SEVERITY_CRITICAL, "EKF variance");`
							`ekf_check_state.last_warn_time = AP_HAL::millis();`
							`}`
							`failsafe_ekf_event();`
							`}`
								Timing hysteresis governing PNT-health messaging.

4	ArduCopter/fence.cpp	10-14	Copter::fence_checks_async() (rate gate)	Async IO callback	AP_HAL::timeout_expired(last_check_ms, now, 40U); AP_HAL::millis()	Rate-limits geofence checking to 25 Hz.	`const uint32_t now = AP_HAL::millis();`
							
							`if (!AP_HAL::timeout_expired(fence_breaches.last_check_ms, now, 40U)) { // 25Hz update rate`
							`return;`
							`}`
								[MULTI-CATEGORY trigger] same function catalogued in Table 4 #3 (positioning).

Table 6a — Indirect Timing dependencies (Layer 1 + Layer 2)

Layer 1 — Direct calls / called-by with typed dependency

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

1	Copter::failsafe_check()	AP_HAL::micros(); scheduler.ticks(); motors->output_min()	1 kHz timer ISR (HAL register_timer_failsafe)	Function call	`void Copter::failsafe_check()`
					`{`
					`  uint32_t tnow = AP_HAL::micros();`
					
					`  const uint16_t ticks = scheduler.ticks();`
					`  if (ticks != failsafe_last_ticks) {`
					`  // the main loop is running, all is OK`
					`  failsafe_last_ticks = ticks;`
					`  failsafe_last_timestamp = tnow;`
					`  if (in_failsafe) {`

2	Plane::failsafe_check()	micros(); scheduler.ticks(); afs/output	1 kHz timer ISR	Function call	`void Plane::failsafe_check(void)`
					`{`
					`  static uint16_t last_ticks;`
					`  static uint32_t last_timestamp;`
					`  static bool in_failsafe;`
					`  uint32_t tnow = micros();`
					
					`  const uint16_t ticks = scheduler.ticks();`
					`  if (ticks != last_ticks) {`
					`  // the main loop is running, all is OK`

Ref #	Component / Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

3	Copter::ekf_check() warn throttle	AP_HAL::millis(); gcs().send_text()	within Copter::ekf_check() bad-variance path	Function call	`LOGGER_WRITE_ERROR(LogErrorSubsystem::EKFCHECK, LogErrorCode::EKFCHECK_BAD_VARIANCE);`
					`// send message to gcs`
					`if ((AP_HAL::millis() - ekf_check_state.last_warn_time) > EKF_CHECK_WARNING_TIME) {`
					`gcs().send_text(MAV_SEVERITY_CRITICAL,"EKF variance");`
					`ekf_check_state.last_warn_time = AP_HAL::millis();`
					`}`
					`failsafe_ekf_event();`
					`}`

4	Copter::fence_checks_async() rate gate	AP_HAL::timeout_expired(); AP_HAL::millis()	IO async callback	Function call	`const uint32_t now = AP_HAL::millis();`
					`if (!AP_HAL::timeout_expired(fence_breaches.last_check_ms, now, 40U)) { // 25Hz update rate`
					`return;`
					`}`

Layer 2 — Full transitive chain to final consumer (hop count)

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
1	Copter::failsafe_check()	scheduler.ticks()/micros() → failsafe_check() → in_failsafe → motors->output_min()	AP_Motors	**2**	[DUPLICATED] timing watchdog.
2	Plane::failsafe_check()	ticks/micros() → failsafe_check() → AFS/failsafe	SRV_Channels / AP_Motors	**2**	[DUPLICATED] timing watchdog.
3	ekf_check warn throttle	AP_HAL::millis() hysteresis → ekf_check() warn → gcs().send_text()	GCS operator	**2**	30 s hysteresis.
4	fence rate gate	timeout_expired(40 ms) → fence_checks_async() gate → fence.check() cadence	fence check rate (25 Hz)	**2**	[MULTI-CATEGORY] also Table 4 #3.